WEEKLY PROGRESS REPORT

Project 5: Crop and Weed Detection using AI

Name: Shrrivathsan S _____	**Domain:** Data Science and Machine Learning	**Date:** 13/6/2026	**Week Ending:** 04

I. Overview

Agriculture is one of the most fundamental sectors of human civilization, yet it faces persistent threats from weeds — unwanted plants that compete directly with cultivated crops for nutrients, sunlight, water, and space. Left uncontrolled, weeds can reduce crop yields by 20–80% depending on the crop type and weed density. Traditional weed management relies heavily on broad-spectrum herbicide application, which is both economically costly and environmentally damaging due to chemical runoff and soil degradation.

This weekly report documents the progress made during Week 1 of Project 5: Crop and Weed Detection — an artificial intelligence initiative aimed at building a precision weed-detection system using deep learning and computer vision. The ultimate goal is to enable smart agricultural robots or spraying drones to identify and target only weed plants with pesticide, leaving the crop untouched. This targeted approach promises to significantly reduce chemical usage, lower production costs, and improve environmental sustainability.

During this reporting week, the team concentrated on the foundational data pipeline: field image acquisition, dataset cleaning, image preprocessing, data augmentation, and manual YOLO-format annotation of 1300 images. These steps are critical prerequisites before any machine learning model can be trained.

II. Project Background & Problem Statement

2.1 The Weed Problem in Agriculture

Weeds are non-cultivated plant species that grow alongside intentional crops and aggressively compete for the same environmental resources. In sesame cultivation — the crop type used in this project — common weed species include grasses, broadleaf plants, and sedges that thrive in similar soil and climate conditions. Farmers typically combat weeds through manual weeding, mechanical cultivation, or chemical herbicide spraying. Each method carries trade-offs: manual and mechanical methods are labour-intensive, while chemical methods introduce risks of pesticide residue on edible produce and ecological harm.

2.2 Limitations of Conventional Pesticide Spraying

Conventional spraying systems operate on a blanket approach — the entire field or row is sprayed regardless of whether weeds are actually present. This results in a substantial waste of chemical inputs, unnecessary exposure of the crop plant to herbicide, increased farming costs, and contamination of surrounding soil and water bodies. Regulatory pressure from food safety authorities is also tightening restrictions on permissible herbicide residue levels on consumable produce, making targeted spraying an increasingly important capability for modern farms.

2.3 Proposed AI-Based Solution

> **Project Aim**
> To develop a computer vision model capable of detecting and classifying sesame crop plants and weed species from field images at 512×512 resolution, enabling a precision spraying system to apply herbicide only to identified weed locations — reducing pesticide use, crop contamination, and environmental impact.

The proposed solution uses the YOLO (You Only Look Once) object detection architecture to perform real-time, single-pass detection of both crop and weed instances within a field image. Each detected object is enclosed in a bounding box labelled either 'crop' or 'weed', along with a confidence score. A downstream control system can then map bounding box coordinates to physical spray nozzle positions on a robotic platform.

III. Dataset Description

3.1 Dataset Overview

The dataset constructed for this project comprises 1300 annotated images of sesame crop fields, covering multiple weed species commonly found in sesame cultivation regions. Each image is a 512×512 three-channel (RGB) colour image, and corresponding annotation files are stored in YOLO format — one text file per image containing class index and normalised bounding box coordinates (centre-x, centre-y, width, height).

Parameter	Value
Total Images (Final)	1300
Image Dimensions	512 × 512 × 3 (RGB)
Annotation Format	YOLO (.txt, normalised coordinates)
Number of Classes	2 — Crop (Sesame), Weed
Original Raw Images	589 field photographs
After Cleaning	546 usable images
Augmentation Multiplier	~2.4× (546 → 1300)
Image Capture Device	DSLR / Smartphone Camera
Original Resolution	4000 × 3000 pixels

3.2 Class Definitions

- Crop (Class 0) — Sesamum indicum (sesame) plants at various growth stages, identifiable by their elongated stem, opposing leaf pairs, and distinctive pod structure.
- Weed (Class 1) — Any non-sesame plant species present within the field boundary, including grass-type monocots, broadleaf dicot weeds, and sedge species.

3.3 YOLO Annotation Format

Each annotation file follows the YOLO label format, where each line represents one object instance:

<class_index> <x_centre> <y_centre> <width> <height>

All coordinate values are normalised relative to image dimensions (range 0.0–1.0), making labels resolution-independent. A class index of 0 denotes crop and 1 denotes weed. Multiple objects per image are represented by multiple lines in the label file.

IV. Achievements This Week

4.1 Phase 1 — Field Data Collection

The data collection phase involved physically visiting sesame cultivation fields to capture images representing the natural co-occurrence of crop and weed plants. Images were taken under varying lighting conditions (morning, afternoon, overcast) and at multiple growth stages to ensure model generalisability. A total of 589 raw photographs were acquired across multiple field visits using DSLR cameras and high-resolution smartphone cameras.

- Coordinated field visits to active sesame cultivation areas across multiple sites.
- Captured images from different angles — top-down, 45-degree oblique, and eye-level — to represent real drone and ground-robot perspectives.
- Ensured coverage of early-stage and mature weed species to improve detection robustness.
- Documented metadata for each image batch including capture date, field location, lighting conditions, and approximate weed density.

4.2 Phase 2 — Dataset Cleaning

Raw field photographs are rarely suitable for direct use in model training. After collection, the full set of 589 images was reviewed manually and programmatically to remove poor-quality samples. This quality control step is essential because noise in training data directly degrades model accuracy and increases convergence time.

- Removed 43 images that were motion-blurred, heavily overexposed, or underexposed beyond usable contrast thresholds.
- Eliminated duplicate frames captured in rapid burst mode that did not add new scene information to the dataset.
- Discarded images where less than 5% of the frame contained identifiable crop or weed specimens (e.g., images accidentally including only bare soil or irrigation equipment).
- The final clean dataset consisted of 546 high-quality, diverse images suitable for preprocessing and annotation.

4.3 Phase 3 — Image Preprocessing

Raw images captured with DSLRs and smartphones have native resolutions of approximately 4000×3000 pixels. While high resolution preserves detail, feeding such images directly into a deep learning model is computationally prohibitive during training — GPU memory constraints and batch processing speed make this impractical. Preprocessing converted all images to a standardised 512×512×3 format.

- Applied bilinear interpolation-based resizing using OpenCV to reduce images from 4000×3000 to 512×512 pixels while preserving aspect fidelity.
- Converted all images to 3-channel RGB format, discarding any EXIF metadata and alpha channels.
- Verified pixel value ranges (0–255 uint8) and confirmed no channel inversion artefacts from BGR/RGB format swaps.

- Validated that corresponding YOLO label coordinates remained accurate after resizing, since YOLO uses normalised coordinates independent of absolute pixel dimensions.

4.4 Phase 4 — Data Augmentation

With 546 clean images, the raw dataset is insufficient for training a robust deep learning model — industry guidance typically recommends at least 1000–2000 samples per class for YOLO-based detectors. Data augmentation artificially expands dataset diversity by applying controlled transformations to existing images, simulating the variance a real-world model would encounter during deployment.

The Keras ImageDataGenerator library and Albumentations framework were used to generate augmented variants, expanding the dataset from 546 to 1300 images (approximately 2.4× expansion).

Augmentation Techniques Applied:

Augmentation Type	Parameters Used	Purpose
Horizontal Flip	p = 0.5	Simulates mirrored field orientation
Random Rotation	±15 degrees	Accounts for camera tilt and drone yaw
Brightness Adjustment	Range: 0.7× – 1.3× brightness	Handles different lighting conditions
Gaussian Noise	Variance: 10–50	Replicates sensor noise in low-light
Random Zoom	Scale: 0.85× – 1.15×	Simulates varying camera altitudes
CLAHE	Clip limit = 2.0	Enhances local contrast for weed features

Albumentations was preferred for spatial augmentations (flip, rotation, zoom) because it automatically adjusts bounding box coordinates to match the transformed image geometry, ensuring label-image alignment is preserved throughout augmentation.

4.5 Phase 5 — Manual YOLO Annotation

Manual annotation is the most labour-intensive phase of the data preparation pipeline. Each of the 1300 images required human reviewers to draw bounding boxes around every individual crop plant and weed specimen visible in the frame. The LabelImg annotation tool was used, configured to export labels in YOLO format.

- Annotators were briefed with a visual taxonomy guide distinguishing sesame crop characteristics from common weed morphologies.
- Each image was reviewed by a primary annotator and cross-checked by a second reviewer to maintain bounding box accuracy and class assignment consistency.
- Difficult edge cases (partially occluded plants, overlapping specimens, very small seedlings) were flagged and discussed as a team before final labelling.
- Quality checks confirmed that all label files have valid normalised coordinates within the 0.0–1.0 range and that class indices match the defined schema (0 = crop, 1 = weed).

V. Challenges Encountered

5.1 Field Data Acquisition Difficulties

Capturing diverse, high-quality images in an active agricultural field presented several logistical and technical challenges. Weather unpredictability meant some field visits resulted in images taken under sub-optimal lighting or after recent rainfall, which changed leaf colour and soil contrast significantly. Gaining consistent access to the same field plots at different times of day required coordination with farm owners. Additionally, varying weed density across different zones of the same field meant that some images were heavily weed-dominated while others contained very few weed instances, introducing class imbalance considerations.

5.2 Annotation Consistency and Scale

Manual bounding box annotation across 1300 images is an extremely time-intensive process that introduces opportunities for human error. Ensuring consistent bounding box tightness — neither too loose (including background) nor too tight (cutting off plant edges) — required ongoing calibration between annotators. Small seedling-stage weed plants that closely resemble juvenile sesame plants were particularly difficult to classify with confidence, requiring botany reference material to resolve. Maintaining annotation momentum across multiple sessions also proved challenging due to annotator fatigue.

5.3 Class Imbalance Risk

Preliminary analysis of the annotated dataset revealed that weed instances outnumber crop instances by an approximate ratio of 3:1 across the full set of labels. This imbalance, if unaddressed, may cause the trained model to exhibit a bias toward predicting the weed class more frequently, potentially reducing crop detection recall. Strategies under consideration include class-weighted loss functions during training, oversampling of crop-dominant images during batching, and synthetic augmentation specifically targeted at crop instances.

5.4 Computational Resource Constraints

Resizing 1300 images from 4000×3000 to 512×512 and running augmentation pipelines generates significant I/O load. While preprocessing itself is manageable on a standard workstation, projecting forward to model training reveals that training a YOLOv8 model for 50+ epochs on 1300 images will require GPU acceleration. The team is currently assessing access to cloud GPU resources (Google Colab Pro, AWS EC2 GPU instances) to ensure training can proceed efficiently.

VI. Learning Resources Utilised

6.1 Object Detection Theory

- Studied the original YOLO papers (v1 through v8) to understand the evolution of single-stage detection architectures, anchor box design, and the shift to anchor-free detection in YOLOv8.
- Reviewed the concept of Intersection over Union (IoU) as the primary metric for measuring bounding box quality during training, and its role in Non-Maximum Suppression (NMS) for removing duplicate detections.
- Examined how mean Average Precision (mAP) at IoU thresholds of 0.5 and 0.5:0.95 is calculated and interpreted as a holistic model quality metric.

6.2 Data Preparation Tools

- Keras ImageDataGenerator: Explored the official TensorFlow/Keras documentation for augmentation parameters, flow() method, and batch generation behaviour.
- Albumentations: Followed the official tutorials on BboxParams and YOLO-format support to ensure spatial augmentations correctly transform label coordinates.
- LabelImg: Used the GitHub documentation and video walkthroughs to configure YOLO export mode and set up efficient keyboard shortcuts for faster annotation.
- Roboflow: Investigated Roboflow's dataset management platform as a potential pipeline tool for augmentation, version control, and YOLO export.

6.3 Computer Vision Libraries

- OpenCV: Practised image reading, resizing (cv2.resize), colour space conversion (cv2.cvtColor), and batch file processing using Python pathlib.
- Pillow (PIL): Used for format conversion, metadata stripping, and verifying image channel consistency across the dataset.
- NumPy: Applied for pixel array manipulation, normalisation verification, and statistical analysis of dataset characteristics.

6.4 Agricultural Reference Material

- Consulted plant morphology guides for sesame (Sesamum indicum) to help annotators distinguish young crop seedlings from similar-looking weed species.
- Reviewed published research papers on deep learning-based weed detection in row crops, including studies on corn, soybean, and cotton, to understand applicable model architectures and benchmark accuracy levels.

VII. Next Week's Goals

7.1 Dataset Finalisation

1. Complete the remaining annotation backlog — approximately 150 images still require second-pass review and quality verification before the dataset is declared production-ready.
2. Address class imbalance by identifying all images with fewer than 3 crop instances and prioritising additional augmentation of those samples.
3. Generate a final dataset statistics report including per-class object counts, average bounding box sizes, and image diversity metrics.

7.2 Training Environment Setup

1. Configure the YOLOv8 training environment — install Ultralytics library, validate GPU availability (CUDA/cuDNN), and perform a dry-run training pass on a small data subset to confirm pipeline integrity.
2. Prepare the dataset.yaml configuration file specifying training/validation/test split paths and class name mapping.
3. Implement the 80/10/10 train-validation-test split with stratified sampling to ensure proportional class representation across all splits.

7.3 Model Training & Validation

1. Launch initial YOLOv8-nano (yolov8n) training run for 50 epochs with default hyperparameters as a baseline experiment, targeting a minimum mAP@50 of 0.70 on the validation set.
2. Log training curves (loss, precision, recall, mAP) using Weights & Biases or TensorBoard for real-time monitoring.
3. If baseline performance is satisfactory, proceed with hyperparameter tuning experiments adjusting learning rate, batch size, and augmentation intensity.

7.4 Evaluation & Reporting

- Generate confusion matrices, precision-recall curves, and detection visualisations on held-out test images.
- Analyse per-class performance — separately report crop detection accuracy versus weed detection accuracy to identify which class the model struggles with.
- Document all experimental results and prepare a comparative table for next week's progress report.

VIII. Technical Approach — Model Architecture

8.1 Why YOLOv8?

YOLOv8 (You Only Look Once, version 8) was selected as the primary detection architecture for this project based on three key criteria: speed, accuracy, and community adoption. Unlike two-stage detectors such as Faster R-CNN that generate region proposals before classification, YOLO processes the entire image in a single forward pass, producing bounding box predictions and class probabilities

simultaneously. This makes YOLO highly suitable for potential real-time deployment on drone or robotic sprayer hardware where inference latency directly impacts operational efficiency.

YOLOv8, released by Ultralytics in 2023, introduced an anchor-free detection head, a redesigned CSPDarknet backbone, and improved data augmentation built into the training loop. These improvements translate to better small-object detection performance — a critical requirement for identifying small or partially occluded weed seedlings in field images.

8.2 Model Variants Under Consideration

Model	Parameters	Speed (ms)	mAP@50 (COCO)	Suitability
YOLOv8n (Nano)	3.2M	1.47ms	37.3	Edge devices, fast prototyping
YOLOv8s (Small)	11.2M	2.33ms	44.9	Balanced — recommended for this project
YOLOv8m (Medium)	25.9M	5.09ms	50.2	High accuracy, needs stronger GPU
YOLOv8l (Large)	43.7M	8.06ms	52.9	Research benchmark only

Training will begin with YOLOv8n for rapid iteration and baseline establishment, before progressing to YOLOv8s as the primary production candidate. Model selection will be guided by the accuracy-latency trade-off requirements of the target spraying platform.

8.3 Training Configuration

Hyperparameter	Planned Value	Rationale
Image size	512×512	Matches dataset resolution; preserves spatial detail
Batch size	16	Memory-efficient for 8GB VRAM GPU
Epochs	50 (max 100)	Sufficient for convergence on 1300 images
Learning rate	0.01 → 0.0001	Cosine decay schedule for stable convergence
IoU threshold	0.5	Standard YOLO NMS threshold
Confidence	0.25 (train)	Permissive threshold during training
Early stopping	Patience = 15	Prevents overfitting to training data

IX. Additional Comments

Week 1 has established a solid and well-documented data foundation for the Crop and Weed Detection project. The team has successfully navigated the demanding process of field data collection, rigorous quality control, systematic preprocessing, strategic augmentation, and painstaking manual annotation. The 1300-image labelled dataset is on track to be fully finalised by the end of Week 2, at which point model training can commence without delay.

The decision to use Albumentations for bounding-box-aware augmentation proved to be a technically sound choice, as it eliminates the label-image misalignment errors that can arise when spatial transformations are applied to images without updating corresponding YOLO coordinate files. This detail, while often overlooked in simpler projects, is essential for maintaining training data integrity.

From a project management perspective, the team is progressing on schedule. The class imbalance issue identified during annotation analysis has been proactively logged and will be addressed during model training through weighted loss and targeted augmentation strategies. Broader collaboration with domain experts in agronomy is being planned to improve annotation quality for ambiguous plant species boundary cases.

> **Overall Status**
> ON TRACK — Data pipeline 90% complete. Model training scheduled to begin in Week 2. No blockers currently identified. GPU access arrangements in progress.

X. Project Source Code:

```
"""
================================================================
===
  Project 5: Crop and Weed Detection using YOLOv8
  Dataset: 1300 images (512x512) | Labels: YOLO format
================================================================
===
"""
```

```
# ——— 0. DEPENDENCIES ———————————————————————————————

# pip install ultralytics opencv-python tensorflow pillow scikit-learn albumentations


import os

import shutil

import random

from pathlib import Path


import cv2

import numpy as np

from PIL import Image



#
═══════════════════════════════════════════════════════════════════════════

#  STEP 1 — Image Preprocessing

#  Resize raw field images from 4000x3000 → 512x512

#
═══════════════════════════════════════════════════════════════════════════


def preprocess_images(input_dir: str, output_dir: str, target_size: tuple = (512, 512)):

    """

    Resize all images in input_dir to target_size and save to output_dir.
```

```
    Args:
        input_dir  : Folder containing raw images (.jpg/.png)
        output_dir : Folder to save resized images
        target_size: (width, height) for output images
    """
    os.makedirs(output_dir, exist_ok=True)
    supported = (".jpg", ".jpeg", ".png", ".bmp")
    images = [f for f in Path(input_dir).iterdir() if f.suffix.lower() in supported]
    print(f"[Preprocessing] Found {len(images)} images in '{input_dir}'")

    for img_path in images:
        img = cv2.imread(str(img_path))
        if img is None:
            print(f"  [WARN] Skipping unreadable file: {img_path.name}")
            continue
        resized = cv2.resize(img, target_size, interpolation=cv2.INTER_AREA)
        out_path = os.path.join(output_dir, img_path.name)
        cv2.imwrite(out_path, resized)

    print(f"[Preprocessing] Done. {len(images)} images saved to '{output_dir}'")


#
```

```
# STEP 2 — Data Augmentation
# Expand 546 clean images → 1300 using Keras / Albumentations
#
================================================================
================

def augment_dataset_keras(input_dir: str, output_dir: str, target_count: int =
1300):
    """
    Use Keras ImageDataGenerator to augment images.
    Each image is saved multiple times with random transformations.
    NOTE: YOLO labels must be updated separately if using spatial transforms.
    """
    import tensorflow as tf
    from tensorflow.keras.preprocessing.image import ImageDataGenerator,
img_to_array, load_img, save_img


    os.makedirs(output_dir, exist_ok=True)
    datagen = ImageDataGenerator(
        rotation_range=20,
        width_shift_range=0.1,
        height_shift_range=0.1,
        zoom_range=0.15,
        horizontal_flip=True,
        brightness_range=[0.7, 1.3],
        fill_mode="nearest"
    )
```

```
    images = list(Path(input_dir).glob("*.jpg")) +
list(Path(input_dir).glob("*.png"))
    generated = 0
    idx = 0

    while generated < target_count:
        img_path = images[idx % len(images)]
        img = load_img(str(img_path), target_size=(512, 512))
        x = img_to_array(img)
        x = x.reshape((1,) + x.shape)

        for batch in datagen.flow(x, batch_size=1, save_to_dir=output_dir,
                                  save_prefix="aug", save_format="jpg"):
            generated += 1
            break  # one augmented version per iteration

        idx += 1
        if generated >= target_count:
            break

    print(f"[Augmentation] Generated {generated} images in '{output_dir}'")


def augment_dataset_albumentations(image_path: str, label_path: str,
                                   out_img_dir: str, out_lbl_dir: str,
```

```
                        n_augments: int = 3):
    """
    Augment a single image + YOLO label while keeping bounding boxes valid.
    Uses Albumentations (bbox-aware augmentation).

    Args:
        image_path  : Path to a single 512x512 image
        label_path  : Corresponding YOLO .txt label file
        out_img_dir : Directory to save augmented images
        out_lbl_dir : Directory to save augmented labels
        n_augments  : Number of augmented copies to generate
    """
    import albumentations as A

    os.makedirs(out_img_dir, exist_ok=True)
    os.makedirs(out_lbl_dir, exist_ok=True)

    transform = A.Compose([
        A.HorizontalFlip(p=0.5),
        A.RandomBrightnessContrast(p=0.4),
        A.Rotate(limit=15, p=0.5),
        A.GaussNoise(var_limit=(10, 50), p=0.3),
        A.CLAHE(p=0.3),
    ], bbox_params=A.BboxParams(format="yolo",
label_fields=["class_labels"]))
```

```
    image = cv2.imread(image_path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    bboxes, class_labels = [], []
    if os.path.exists(label_path):
        with open(label_path) as f:
            for line in f:
                parts = line.strip().split()
                class_labels.append(int(parts[0]))
                bboxes.append([float(v) for v in parts[1:5]])

    stem = Path(image_path).stem
    for i in range(n_augments):
        result = transform(image=image, bboxes=bboxes,
class_labels=class_labels)
        aug_img = cv2.cvtColor(result["image"], cv2.COLOR_RGB2BGR)
        aug_name = f"{stem}_aug{i}"
        cv2.imwrite(os.path.join(out_img_dir, aug_name + ".jpg"), aug_img)

        with open(os.path.join(out_lbl_dir, aug_name + ".txt"), "w") as lf:
            for cls, bbox in zip(result["class_labels"], result["bboxes"]):
                lf.write(f"{cls} {' '.join(f'{v:.6f}' for v in bbox)}\n")

    print(f"[Albumentations] Augmented '{stem}' × {n_augments}")
```

```
#
═══════════════════════════════════════════════════════════════════════
#  STEP 3 — Train/Val/Test Split
#  80% training | 10% validation | 10% test
#
═══════════════════════════════════════════════════════════════════════

def split_dataset(images_dir: str, labels_dir: str, output_dir: str,
                  ratios: tuple = (0.8, 0.1, 0.1), seed: int = 42):
    """
    Split image+label pairs into train / val / test sets for YOLOv8.

    Directory structure produced:
        output_dir/
          images/train/  images/val/  images/test/
          labels/train/  labels/val/  labels/test/
    """
    random.seed(seed)
    images = sorted(Path(images_dir).glob("*.jpg"))
    random.shuffle(images)

    n = len(images)
    n_train = int(n * ratios[0])
    n_val = int(n * ratios[1])
    splits = {
```

```
        "train": images[:n_train],
        "val":   images[n_train:n_train + n_val],
        "test":  images[n_train + n_val:]
    }

    for split, img_list in splits.items():
        img_out = Path(output_dir) / "images" / split
        lbl_out = Path(output_dir) / "labels" / split
        img_out.mkdir(parents=True, exist_ok=True)
        lbl_out.mkdir(parents=True, exist_ok=True)

        for img_path in img_list:
            shutil.copy(img_path, img_out / img_path.name)
            lbl_path = Path(labels_dir) / (img_path.stem + ".txt")
            if lbl_path.exists():
                shutil.copy(lbl_path, lbl_out / lbl_path.name)

        print(f"  [{split:5s}] {len(img_list)} images")

    print(f"[Split] Dataset split complete in '{output_dir}'")

    # Generate dataset.yaml for YOLOv8
    yaml_path = Path(output_dir) / "dataset.yaml"
    yaml_content = f"""
path: {os.path.abspath(output_dir)}
train: images/train
```

```
val: images/val
test: images/test

nc: 2
names:
  0: crop
  1: weed
""".strip()
    yaml_path.write_text(yaml_content)
    print(f"[Split] dataset.yaml written to '{yaml_path}'")
    return str(yaml_path)



#
================================================================
================
#  STEP 4 — Model Training (YOLOv8)
#
================================================================
================


def train_yolov8(yaml_path: str, model_variant: str = "yolov8n.pt",
                 epochs: int = 50, img_size: int = 512,
                 batch: int = 16, project: str = "runs/train"):
    """
    Train a YOLOv8 model on the crop/weed dataset.
```

```
    Args:
        yaml_path     : Path to dataset.yaml
        model_variant : 'yolov8n.pt' (nano), 'yolov8s.pt' (small), etc.
        epochs        : Number of training epochs
        img_size      : Input image size
        batch         : Batch size
        project       : Directory to save training results
    """
    from ultralytics import YOLO


    model = YOLO(model_variant)
    results = model.train(
        data=yaml_path,
        epochs=epochs,
        imgsz=img_size,
        batch=batch,
        project=project,
        name="crop_weed",
        patience=15,       # early stopping
        lr0=0.01,
        lrf=0.0001,
        weight_decay=0.0005,
        mosaic=1.0,        # mosaic augmentation
        degrees=10.0,
        flipud=0.0,
        fliplr=0.5,
```

```
        hsv_h=0.015,
        hsv_s=0.7,
        hsv_v=0.4,
        verbose=True,
        device="0",        # set to 'cpu' if no GPU
    )
    print(f"\n[Training] Best weights saved to: {results.save_dir}")
    return results


#
═══════════════════════════════════════════════════════════════════
═══════════════════
#  STEP 5 — Evaluation
#
═══════════════════════════════════════════════════════════════════
═══════════════════


def evaluate_model(weights_path: str, yaml_path: str, split: str = "test"):
    """
    Evaluate trained model on the test split.
    Prints mAP50, mAP50-95, Precision, Recall.
    """
    from ultralytics import YOLO

    model = YOLO(weights_path)
    metrics = model.val(data=yaml_path, split=split)
```

```
    print("\n" + "="*50)
    print("  MODEL EVALUATION RESULTS")
    print("="*50)
    print(f"  mAP@50       : {metrics.box.map50:.4f}")
    print(f"  mAP@50-95    : {metrics.box.map:.4f}")
    print(f"  Precision    : {metrics.box.mp:.4f}")
    print(f"  Recall       : {metrics.box.mr:.4f}")
    print("="*50)
    return metrics


#
═══════════════════════════════════════════════════════════════════
═══════════════════
#  STEP 6 — Inference on New Images
#
═══════════════════════════════════════════════════════════════════
═══════════════════


def run_inference(weights_path: str, source: str,
                  conf_threshold: float = 0.4, output_dir: str = "results/"):
    """
    Run weed/crop detection on a folder of images or a single image.

    Args:
        weights_path   : Path to trained best.pt weights
```

```
    source         : Image file, folder, or video path
    conf_threshold : Minimum confidence to display a detection
    output_dir     : Where to save annotated result images
"""
from ultralytics import YOLO

model = YOLO(weights_path)
results = model.predict(
    source=source,
    conf=conf_threshold,
    save=True,
    save_dir=output_dir,
    line_width=2,
    show_labels=True,
    show_conf=True,
)

for r in results:
    boxes = r.boxes
    print(f"\nImage: {r.path}")
    print(f"  Detections : {len(boxes)}")
    for box in boxes:
        cls = int(box.cls[0])
        name = r.names[cls]
        conf = float(box.conf[0])
        xyxy = box.xyxy[0].tolist()
```

```
        print(f"  [{name:4s}] conf={conf:.2f}  bbox={[round(v,1) for v in
xyxy]}")


    print(f"\n[Inference] Results saved to '{output_dir}'")
    return results



#
═══════════════════════════════════════════════════════════════════
═══════════════════
#  STEP 7 — Visualise Detections with OpenCV
#
═══════════════════════════════════════════════════════════════════
═══════════════════


def visualise_detections(image_path: str, weights_path: str,
                         conf_threshold: float = 0.4, save_path: str = None):
    """
    Draw bounding boxes on an image and display / save the result.
    Weeds → Red boxes | Crops → Green boxes
    """
    from ultralytics import YOLO

    CLASS_COLORS = {
        "crop": (0, 200, 0),    # Green
        "weed": (0, 0, 220),    # Red (BGR)
    }
```

```
model = YOLO(weights_path)
result = model.predict(source=image_path, conf=conf_threshold)[0]

img = cv2.imread(image_path)
for box in result.boxes:
    x1, y1, x2, y2 = map(int, box.xyxy[0])
    cls_name = result.names[int(box.cls[0])]
    conf = float(box.conf[0])
    color = CLASS_COLORS.get(cls_name, (255, 255, 0))
    cv2.rectangle(img, (x1, y1), (x2, y2), color, 2)
    label = f"{cls_name} {conf:.2f}"
    cv2.putText(img, label, (x1, y1 - 8),
                cv2.FONT_HERSHEY_SIMPLEX, 0.55, color, 2)

if save_path:
    cv2.imwrite(save_path, img)
    print(f"[Visualise] Annotated image saved to '{save_path}'")
else:
    cv2.imshow("Crop & Weed Detection", img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

```
# ═══════════════════════════════════════════════════════════════
#  MAIN PIPELINE
# ═══════════════════════════════════════════════════════════════

if __name__ == "__main__":

    # ── Paths (update to your local paths) ──────────────────────
    RAW_IMAGES_DIR    = "data/raw_images"       # 589 original photos
    CLEAN_IMAGES_DIR  = "data/clean_images"     # 546 after cleaning
    RESIZED_DIR       = "data/resized_512"      # 512x512 resized
    AUGMENTED_IMGS    = "data/augmented/images" # 1300 augmented images
    AUGMENTED_LBLS    = "data/augmented/labels" # corresponding YOLO labels
    DATASET_DIR       = "data/dataset"          # train/val/test split
    WEIGHTS           = "runs/train/crop_weed/weights/best.pt"

    # ── Step 1: Preprocess (resize) ─────────────────────────────
    print("\n" + "─"*55)
    print("  STEP 1 — Preprocessing")
    print("─"*55)
    preprocess_images(CLEAN_IMAGES_DIR, RESIZED_DIR, target_size=(512, 512))
```

```
    # ── Step 2: Augment (Albumentations, bbox-safe) ──────────────
    print("\n" + "═"*55)
    print("  STEP 2 — Data Augmentation")
    print("═"*55)
    labels_dir = "data/labels_clean"
    for img_file in Path(RESIZED_DIR).glob("*.jpg"):
        lbl_file = Path(labels_dir) / (img_file.stem + ".txt")
        augment_dataset_albumentations(
            str(img_file), str(lbl_file),
            AUGMENTED_IMGS, AUGMENTED_LBLS, n_augments=2
        )
    print(f"Dataset expanded using Albumentations augmentation.")


    # ── Step 3: Split dataset
──────────────────────────────────────────────────

    print("\n" + "═"*55)
    print("  STEP 3 — Train / Val / Test Split  (80/10/10)")
    print("═"*55)
    yaml_path = split_dataset(AUGMENTED_IMGS, AUGMENTED_LBLS,
DATASET_DIR)


    # ── Step 4: Train YOLOv8
──────────────────────────────────────────────────

    print("\n" + "═"*55)
    print("  STEP 4 — Model Training (YOLOv8)")
    print("═"*55)
```

```
    train_yolov8(yaml_path, model_variant="yolov8n.pt", epochs=50, batch=16)


    # ── Step 5: Evaluate ──────────────────────────────────────────────
    print("\n" + "═"*55)
    print("  STEP 5 — Evaluation on Test Set")
    print("═"*55)
    evaluate_model(WEIGHTS, yaml_path, split="test")


    # ── Step 6: Inference demo ────────────────────────────────────────
    print("\n" + "═"*55)
    print("  STEP 6 — Inference Demo")
    print("═"*55)
    run_inference(WEIGHTS, source="data/sample_test_images/",
conf_threshold=0.4)
```

```
# ── Step 7: Visualise a single result ──────────────────────────
  print("\n" + "═"*55)
  print("  STEP 7 — Visualisation")
  print("═"*55)
  visualise_detections(
      image_path="data/sample_test_images/test_001.jpg",
      weights_path=WEIGHTS,
      conf_threshold=0.4,
      save_path="results/annotated_test_001.jpg"
  )

  print("\n[Pipeline Complete] Crop & Weed Detection project finished.")
```